International Journal of HIGH PERFORMANCE COMPUTING APPLICATIONS

Special Issue Paper

Checkpointing fine-tuning for accelerating seismic applications in GPUs

The International Journal of High Performance Computing Applications
2025, Vol. 39(6) 784–802

DOI: 10.1177/10943420251340794
journals.sagepub.com/home/hpc
Sage

Thiago Maltempi[1], Sandro Rigo[1], Marcio Pereira[1], Hervé Yviquel[1], Gustavo Leite[1], Orlando Lee[1], Jessé Costa[2,3] and Guido Araujo[1]

Abstract

High-performance computing (HPC) systems are essential to handle computationally intensive tasks in fields such as physics, climate modeling, and seismic analysis. Reverse Time Migration (RTM), a widely used seismic imaging technique for oil reservoir exploration, exemplifies these challenges, requiring vast amounts of memory and extended computation times. RTM relies on checkpointing to store data during forward wave propagation for reuse in the backward phase. However, traditional checkpoint methods are constrained by costly host-GPU data transfers, which limits performance. GPUZIP v2.0 addresses these bottlenecks with several enhancements over its predecessor: (1) A GPU Checkpoint Cache with a least recently used (LRU) policy enables flexible checkpoint storage configurations and efficient prefetching; (2) A redesigned prefetch algorithm further increases cache hit ratios; and (3) The integration of three distinct checkpointing algorithms provides adaptability for diverse application profiles. These advancements allow fine-tuning of RTM and similar applications, significantly improving performance. The experimental results show that GPUZIP v2.0 achieves speedups of up to 5.12×, surpassing the 3.9× achieved by its previous version. GPUZIP v2.0 provides an effective solution for accelerating memory-intensive HPC applications by reducing data transfer overhead and offering customized checkpoint strategies. GPUZIP is publicly available via GitHub.

Keywords

High-performance computing, data compression, reverse time migration, prefetching, checkpointing

1. Introduction

High-performance computing (HPC) systems are essential for advancing scientific research and solving complex computational problems in fields ranging from physics and climate modeling to bioinformatics and seismic analysis. These applications require intensive computational resources to handle large datasets and perform large-scale simulations. To meet this demand, nearly all modern HPC systems have adopted GPUs for acceleration, capitalizing on their highly parallel architecture to perform calculations that would otherwise be infeasible on traditional CPUs alone.

This research focuses on the Reverse Time Migration (RTM) application as a case study application. RTM is a seismic imaging technique introduced by Baysal et al. (1983) that generates high-resolution subsurface images from seismic data, mainly used in the exploration of oil reservoirs. RTM is computationally intensive, requiring hundreds of gigabytes of memory and extended computation times that can span several days.

[1]Instituto de Computação - Universidade Estadual de Campinas (UNICAMP), Campinas, Brazil
[2]Faculty of Geophysics - Pará Federal University (UFPA), Belém, Brazil
[3]INCT-Petroleum Geophysics, Salvador, Brazil

Corresponding authors:
Thiago Maltempi, Universidade Estadual de Campinas (UNICAMP), Instituto de Computação, Laboratório de Sistemas de Computação, IC 3 / 3,5 – R. Saturnino de Brito, 573, Cidade Universitária, Campinas – SP, 13083-852, Brazil.
Email: maltempi@ic.unicamp.br

Sandro Rigo, Universidade Estadual de Campinas (UNICAMP), Instituto de Computação, Laboratório de Sistemas de Computação, Av. Albert Einstein, 1251 - Room 42, Cidade Universitária, Campinas – SP, 13083-852, Brazil.
Email: srigo@unicamp.br

RTM uses seismic survey data as input, where sound waves produced by an air gun (or vibroseis truck in land surveys) reflect off subsurface structures and are recorded by hydrophones (or geophones on land). RTM simulates wave propagation through a two-phase approach. In the forward phase, wave propagation is computed in a forward-time direction, from the source to the recording devices. In the backward phase, simulation begins at the recording devices, using survey data from the source, but following a reverse time step. The forward and backward simulated wave fields are then correlated at each spatial grid point to produce the final subsurface image.

Checkpointing plays a pivotal role in this method, preserving the data of the GPU kernel from the forward phase for reuse in the backward phase and preventing recomputation. However, storing data from all time steps is infeasible due to memory constraints, necessitating a balance between memory usage and recomputation. Traditionally, checkpoints are saved in the host memory and retrieved as needed, but this introduces a major bottleneck, as data transfer between host and GPU memory is costly and can severely impact performance.

In previous work, GPUZIP v1.0 (Maltempi et al., 2024a) was introduced as an effective solution to mitigate the checkpoint data transfer overhead between the GPU memory and the host memory. By combining a prefetching mechanism with GPU-based lossy compression, GPUZIP enables the transfer of compressed data over the PCI-e bus only. This combined approach significantly reduces the volume of Host-to-Device (PCI-e) data transfers, achieving reductions of up to 80× in transfer volume.

This article introduces GPUZIP v2.0, which contains contributions that significantly enhance the flexibility to tailor the checkpoint mechanism to specific application profiles, resulting in improved performance: (1) The GPU data structure has evolved from a *Two-Position GPU Checkpoint Buffer* to a fully associative *GPU Checkpoint Cache* with a least recently used (LRU) policy and a self-contained data synchronization mechanism. (2) This cache enables customization of the number of checkpoints stored, greatly expanding the exploration space for fine-tuning the prefetching mechanism. (3) A redesigned prefetch algorithm leverages the cache configuration to increase hit ratios. (4) An analysis of three distinct checkpoint algorithms integrated into the RTM application using GPUZIP, detailing various configuration options, and demonstrating how GPUZIP can guide optimal setup for each combination of application and checkpoint strategy. And finally (5), we are making GPUZIP publicly available on GitHub (git, 2025).

Furthermore, this article demonstrates that these contributions have further improved performance, with speedups increasing from 3.9× to 5.12× in the current version.

This article is structured as follows: Section 2 provides an updated review of related work. Section 3 introduces the key concepts used as a basis for this investigation, including RTM, the checkpoint paradigm, a description of the checkpointing algorithms utilized, and an overview of GPUZIP v1.0. Section 4 details the new Prefetching mechanism brought in GPUZIP v2.0. Section 5 explores the role of lossy compression techniques in improving overall speedup and outlines the criteria for configuring the compressors used in this study. Section 6 describes the experimental setup, including cluster specifications, datasets, and GPUZIP configurations to ensure reproducibility. Section 7 presents the results of the new experiments, and finally, Section 8 summarizes the findings and discusses future directions for this research.

2. Related work

This section reviews the relevant literature on the techniques discussed in this article. GPUZIP introduces a novel approach by integrating GPU-based lossy compression with the prefetching of checkpoint data from the host to GPU memory. It works seamlessly with different checkpointing algorithms and allows for the exploration of prefetching and compression parameters to fine-tune the checkpoint mechanism to specific application profiles. To the best of our knowledge, prior research has typically treated compression and prefetching as distinct processes, often using different methodologies or focusing on other objectives.

Lossy compression techniques are widely used in scientific computing (Di et al., 2025). Barbosa and Coutinho (2023) compresses the RTM wavefield on the host and then transfers the compressed data to the GPU. Dmitriev et al. (2022) and Huang et al. (2023b) investigate GPU-based lossy compressors for RTM applications, focusing on reducing data storage and evaluating the trade-off between compression quality and efficiency. It should be noted that Dmitriev et al. (2022) employs cuZFP and NVIDIA Bitcomp lossy compressors, which are also analyzed in this work.

In compression of checkpoint data, Margetis et al. (2021, 2023) apply CPU-based lossy compression to reduce checkpoint storage requirements. Kukreja et al. (2019) uses GPU-based compression for checkpoints to save storage space and enable storing more checkpoints, resulting in faster solver execution. Shen et al. (2022, 2023) explore GPU-based compressors to reduce data transfer bottlenecks between the GPU and the host. These studies demonstrate how compression techniques can improve GPU-to-host data transfer performance without compromising the resulting quality.

In the context of memory management for more efficient data transfer between the host and GPUs, Rigon et al. (2024) explores asynchronous prefetching and Unified Memory techniques to optimize data movement for RTM. Their research showed a performance improvement of up to 62% by enhancing data transfer efficiency between the host and the GPU. Although their work does not focus on checkpointing, it

highlights the substantial efficiency gains achievable through prefetching host-to-GPU data for this kind of application.

More specifically in the prefetching of checkpointing data, (Maurya et al., 2023a, 2023b) introduced a multilevel storage prefetching mechanism using the general-purpose checkpointing library VELOC. This mechanism schedules data prefetching when the `SAVE` checkpoint action is invoked. In contrast, GPUZIP proactively schedules prefetch actions in advance before solver execution begins. This proactive approach allows GPUZIP's prefetch algorithm to be flexible enough to support various checkpoint libraries, regardless of when the `SAVE` or `RESTORE` actions are activated during the solver's execution. Moreover, it is simple to support various compression libraries through the GPUZIP API, allowing users to fine-tune the checkpoint mechanism to meet specific application needs.

3. Background

This section introduces the main concepts required to understand the methodology adopted in this article. A brief description of the previous version of GPUZIP (Maltempi et al., 2024a) is also included to make this article more self-contained and clarify the novel contributions.

3.1. Reverse time migration

Reverse Time Migration (RTM) (Baysal et al., 1983) is a high-resolution seismic imaging technique extensively used to explore oil, natural gas and minerals. The process starts with seismic surveys, which capture seismic data from an acoustic source. In land surveys, this source is typically a vibrator hammer mounted on a truck, while at sea, it is an air gun deployed from a ship; hence, geophones or hydrophones record the reflected waves, generating the seismic data.

RTM implementations typically use the Finite Difference Method (FDM), which employs stencil computation to solve the seismic wave equation on a spatial grid. This method approximates derivatives using finite differences, updating grid points iteratively over discrete time steps to simulate wave propagation. Each update relies on the information from the previous timestep to calculate new values.

RTM operates through a two-phase process. First, the *forward phase* computes the wavefield that travels from the air gun (source) to the hydrophones (receivers), as shown in the blue arches in Figure 1. The *backward phase* propagates the recorded signals backward in time by solving the adjoint wave equation, starting from the receiver and propagating back towards the source (green arches in Figure 1). RTM correlates every backward-in-time step with the corresponding forward-in-time step (red dots in Figure 1) and, from this process, generates a migrated image. Geophysicists then use this image to guide the (expensive) decision to drill an oil reservoir.

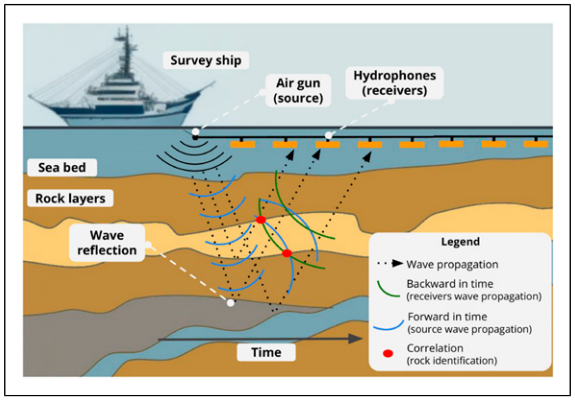

Figure 1. Seismic survey.

3.2. Checkpointing

Checkpointing is a broad concept often associated with fault tolerance in computing. In applications that use reverse-in-time computation, such as RTM (see Subsection 3.1), checkpointing serves a different purpose: it saves computational states during the forward phase, which can then be rematerialized in the backward phase. This process helps avoid recomputation and improves overall performance.

Figure 2 provides an example of checkpointing in action. At the beginning ①, the process proceeds `FORWARD`

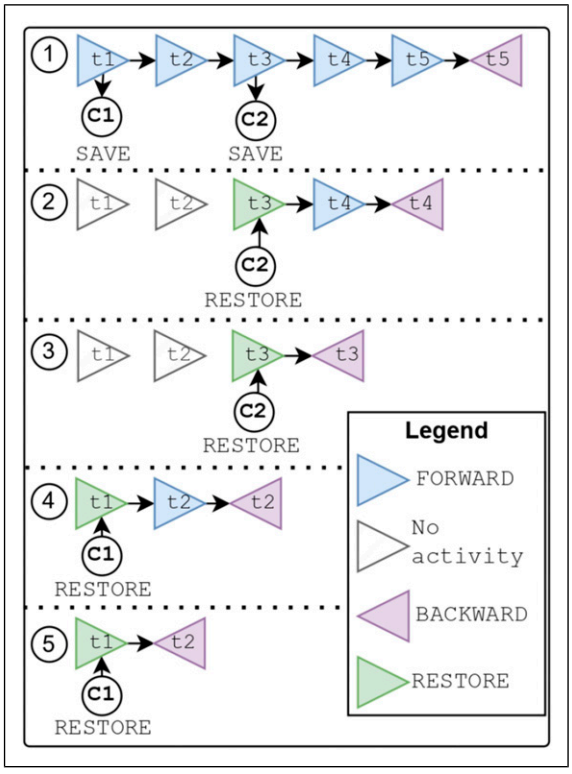

Figure 2. Example of forward and backward checkpointing in adjoint computation with five timesteps and two checkpoints.

through each timestep, starting from timestep `t1` and saving the checkpoint (C1). Then it moves forward, computing timesteps `t2` and `t3`, where the checkpoint (C1) is saved. The forward computation continues until timestep `t5`, after which the `BACKWARD` computation for timestep `t5` is performed using data from the forward phase.

Subsequently, at ②, if no checkpointing was used, the forward computation from timesteps `t1` to `t4` would have to be recomputed. However, since the checkpoint (C2) was saved in the timestep `t3`, the process can restore (C2) and compute `FORWARD` only from `t3` to `t4` before performing `BACKWARD` in `t4`.

The checkpoint (C2) is restored again at ③, allowing for backward computation at `t3`. For timestep `t2` at ④, checkpoint (C1) is restored, and then `FORWARD` to `t2` is computed before moving backward. Finally, at ⑤, (C1) is restored one last time, allowing backward computation at `t1`.

This (short) example just discussed would require 15 computations of `FORWARD` if no checkpointing was used. But in fact, it requires only 7 `FORWARD` recomputations, showing how checkpointing can substantially reduce redundant work.

Ideally, all timesteps from the forward phase of the RTM would be saved to eliminate the need for recomputation. However, this approach is impractical due to the significant storage requirements. For example, storing one checkpoint for the Marmousi3D dataset used in this study requires approximately 500 *MB*. With 6751 timesteps, the total storage demand would reach 3.2 *TB*, which is infeasible. To address this challenge, checkpointing algorithms such as *Revolve* (Griewank and Walther, 2000), *zCut* (under review, 2024), or *Uniform* are employed to balance memory usage and recomputation overhead.

Revolve is a well-known checkpointing algorithm that optimally balances memory use with required recomputations. *Revolve* is integrated into Awave-3D, our in-house RTM implementation, and the *Devito framework* (Louboutin et al., 2019; Luporini et al., 2020), a popular open source toolset for inverse seismic problems. Unlike the simple checkpoint shown above, *Revolve* partitions the computation into intervals and strategically selects a limited number of checkpoints within each interval. This allows for efficient recomputation during the backward phase while minimizing storage requirements.

For example, with the Marmousi3D dataset, *Revolve* requires only seven checkpoints at a time. Initially, the algorithm processes forward from timestep 0 to 3002 without saving checkpoints. Starting at timestep 3003, it saves seven strategically spaced checkpoints up to timestep 6751, enabling efficient forward and backward computation for this interval. After completing the interval, the checkpoints are discarded, as they are no longer needed. *Revolve* then repeats this process for subsequent intervals, saving new checkpoints, and continuing the computation. This method effectively reduces memory usage while minimizing recomputation, making *Revolve* a highly efficient and storage-friendly solution for checkpointing in RTM.

zCut is a dynamic programming algorithm that minimizes the number of checkpoints needed in programs with a forward-backward structure, known as adjoint computation. The program is represented as a directed acyclic graph, called computational graph, where the nodes represent computational kernels and edges correspond to data dependencies between those kernels. The algorithm takes a computational graph as input and the total memory budget of the device. It outputs a series of nodes that should be saved into a checkpoint pool. Nodes not saved during the forward phase must be recomputed from the most recent checkpoint during the backward phase. The main insight of *zCut* is that the overhead of saving and restoring data dominates the execution time. Therefore, it minimizes the amount of data that flows between the device and the checkpoint pool while keeping as much data alive in the device as possible without surpassing its budget.

Uniform by contrast, is a simpler checkpointing algorithm that resembles the example shown in Figure 2. Users specify the maximum number of checkpoints based on available storage, and the algorithm calculates a uniform interval for saving checkpoints. For example, with 6751 timesteps and a storage limit of 100 checkpoints, *Uniform* saves a checkpoint every 67 timesteps.

Despite their differences, the three checkpointing algorithms— *Revolve*, *zCut*, and *Uniform*— share common characteristics. They are deterministic, meaning identical inputs and timesteps always produce the same checkpoint sequence. Additionally, they rely on a central control loop to guide program actions (`BACKWARD`, `FORWARD`, `SAVE`, `RESTORETERMINATE`) for each timestep, ensuring systematic and predictable execution.

3.3. GPUZIP's prefetching mechanism v1.0 Overview

In our previous work, GPUZIP (Maltempi et al., 2024a) introduced a prefetching mechanism as part of its solution, which consists of a *Two-Position GPU Checkpoint Buffer* (referred to as *GPU Buffer*) and the "Prefetch Setup Algorithm".

The *GPU Buffer* serves as an intermediary between the GPU kernels and the *Checkpoint Pool* located in host memory. GPU kernels interact with the buffer to save and restore checkpoint data (see Figure 3 ① and ②). Checkpoints stored in the *GPU Buffer* are temporary copies of those in the *Checkpoint Pool* and can be discarded as needed. During a `RESTORE` operation, GPUZIP first checks if the requested checkpoint is available in the *GPU Buffer*. If it is found, a "Buffer Hit" occurs, which is ideal as it avoids the high cost of transferring data from host memory to GPU memory. Conversely, if the checkpoint is not available in the GPU Buffer, GPUZIP fetches it from host

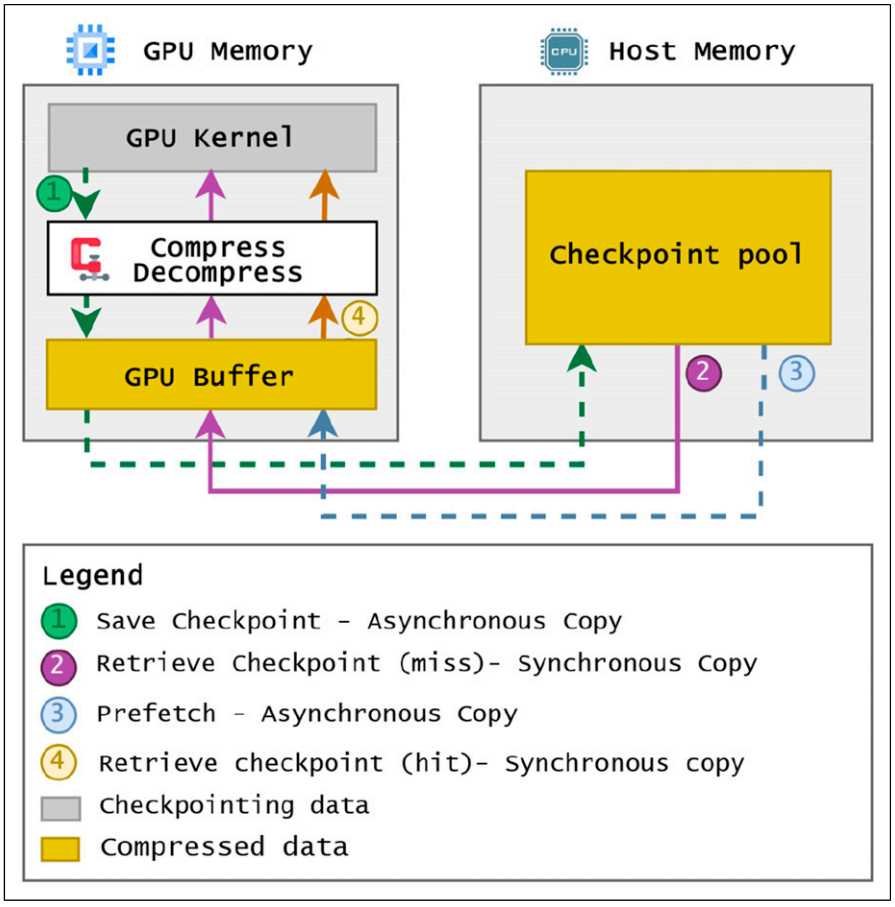

Figure 3. GPUZIP v1.0's memory architecture.

memory, loads it into the GPU Buffer, and rematerializes the checkpoint. This scenario, known as a "Buffer Miss," is less desirable due to the associated transfer overhead.

The deterministic nature and efficiency of the *Revolve* Algorithm allowed the development of the "Prefetch Setup Algorithm". This algorithm simulates the behavior of the *GPU Buffer* by performing a dry run of the *Revolve* process. During this simulation, a "Buffer Miss" triggers the algorithm to inspect the most recent `RESTORE` operation that was not followed by a `SAVE` action. This inspection identifies an empty position in the *GPU Buffer*, enabling the algorithm to optimize prefetching and improve overall efficiency. Suppose that there is no space available in the GPU buffer. In that case, this situation is termed an "Unavoidable Miss", meaning that GPUZIP will need to retrieve the checkpoint from the *Checkpoint Pool* via PCIe during actual RTM execution.

During actual RTM execution, GPUZIP adheres to the prefetch schedule outlined by the "Prefetch Setup Algorithm". It performs asynchronous data transfers from the *Checkpoint Pool* to the *GPU Buffer* (host to device) in advance, ensuring that the data is available when needed. When the *Revolve* algorithm requests a checkpoint that has been prefetched, it results in a "Buffer Hit" (See Figure 3 ③).

The GPUZIP Prefetching Mechanism successfully avoided approximately 75% of the buffer misses in previous experiments. This led to a performance speedup of 1.57× when using prefetching alone (without compression) and a speedup of 3.9× when combining Prefetch with the best data compression, specifically on the Salt dataset (Aminzadeh et al., 1997).

4. The new prefetching mechanism (GPUZIP v2.0)

In many cases, available GPU memory can be utilized to store additional checkpoints, which helps to increase the Buffer Hit rate, thereby reducing blocking PCIe traffic and improving overall performance. As discussed in Section 3.3, the original version (v1.0) of GPUZIP's prefetching mechanism limited the *GPU Buffer* to two positions, which could restrict users from fully leveraging their hardware's capacity to store more checkpoints if possible, according to the application's needs.

The primary enhancement introduced in this work (v2.0) is the ability of users to tune the prefetching mechanism according to their hardware capabilities, application profile, and input data. It all starts by configuring the size of the checkpoint storage in the GPU. This configurable checkpoint storage size significantly impacts the "Prefetch Setup" process, requiring advanced management of multiple checkpoint positions, a more sophisticated retention policy, and an efficient checkpoint lifecycle. It also requires careful control over checkpoint synchronization in the GPU to ensure that checkpoint transfers are completed successfully.

These changes demanded a redesign of the Prefetch Mechanism in GPUZIP. The v2.0 design is introduced as follows: Section 4.1 details the improvements made to what was previously called the *Two-Position GPU Checkpoint Buffer*, now called the *GPU Checkpoint Cache*. This update introduces the concept of a configurable, non-hardcoded capacity for storing checkpoint data in GPU memory. The term *Cache* reflects a more advanced and reliable synchronization mechanism. Subsection 6 discusses how the Prefetch Setup Algorithm adapts to this new cache mechanism in v2.0.

4.1. The GPU checkpoint cache

The *GPU Checkpoint Cache* is a fully associative cache (Hennessy and Patterson, 2017) designed to reside in GPU memory, acting as a bridge between the Checkpoint Pool in host memory and the GPU kernels. Its primary role is to ensure efficient checkpoint data access by storing checkpoints,

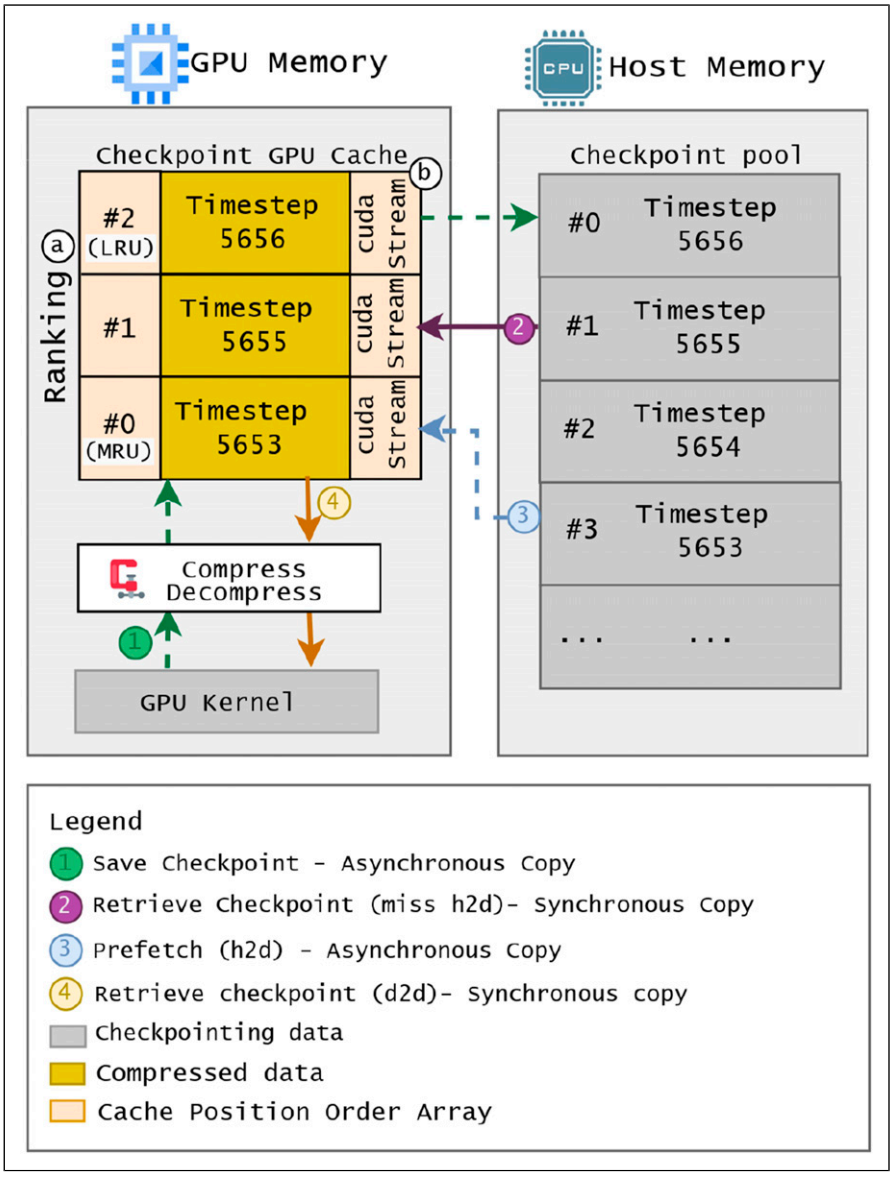

Figure 4. GPUZIP v2.0's memory architecture.

managing their lifecycle, and allowing users to configure the cache size to suit their application requirements.

The cache uses a Least Recently Used (LRU) replacement policy, evicting the least recently accessed checkpoint when additional space is required. Each time a checkpoint is saved or restored, it is promoted to the Most Recently Used (MRU) position, and the remaining entries are reorganized accordingly.

The cache structure consists of an array of data positions, each linked to a rank position and a CUDA stream pointer. "Ranking Array" (Figure 4, element ⓐ) manages checkpoint states, identifying the MRU (rank #0) and LRU (rank #2). For instance, when the cache is full and a new checkpoint is added, the LRU (rank #2) is evicted, the new checkpoint is promoted to MRU (rank #0), and the former MRU moves to rank #1, while former rank #1 shifts to #2.

Each cache position operates with its own independent *cudaStream* (Figure 4, element ⓑ), enabling asynchronous operations. For example, GPUZIP can asynchronously transfer data from rank #2 to host memory (`SAVE` operation in ①) while synchronously retrieving a checkpoint from rank #1 (`RESTORE` with a cache miss in ③). In such scenarios, CUDA synchronization is limited to the active checkpoint (rank #1), ensuring that ongoing asynchronous transfers (rank #0) remain uninterrupted.

This combination of independent streams and the rank array provides a reliable and flexible mechanism for storing and retrieving checkpoints, ensuring efficient data access without data loss due to poor synchronization.

4.2. The prefetch setup algorithm

The Prefetch Setup Algorithm (PSA) predicts all potential cache misses in an application's execution. It creates a schedule indicating when GPUZIP should asynchronously bring the checkpoint data ahead of time from the host memory to the GPU memory, avoiding cache misses.

The essential components of the Prefetch Setup Algorithm are:

(1) **Checkpointing Wrapper**, an API that wraps the checkpointing libraries (*Revolve*, *zCut*, *Uniform*, or any other compatible library) and provides the method "*GetAction()*" that returns the action to be performed (`SAVE`, `RESTORE`, `BACKWARD`, `FORWARD`, `TERMINATE`) and the timestep;
(2) The **Main Loop** that calls the "*GetAction()*" function and dry-runs the actual problem;
(3) **Cache**, an instance of the GPU Checkpoint Cache but in its dummy form, not performing any GPU memory allocation;
(4) **State Recorder** is a class that takes a snapshot of the cache during its iteration, allowing for the cache to be reset to a specific state from that iteration;
(5) The **Prefetch Action Vector (PAV)** is a schedule for the iterations in which a Prefetch action should occur. It is the output of the algorithm.

Algorithm 1 presents a pseudo-code for the *Prefetch Setup Algorithm*. For all iterations of the PSA's main loop, we start by getting the next action to be performed. In case of a `SAVE` action, the *State Recorder* saves a snapshot ① for later restoration. When the *"GetAction()"* function returns a `SAVE` action, the Cache saves the timestep number in the Cache ②. For the `RESTORE` actions, it checks if the asked checkpoint is in the *Cache* - indicating a *Cache Hit*; if true, the cache is touched ③, and the *Cache* promotes the asked checkpoint to *Most Recently Used* and reorganizes all other positions.

In the event of a cache miss, knowing that the cache will sacrifice the LRU item to store the new checkpoint, the algorithm will find the last iteration that touches the *Least Recently Used* in the cache. Once it finds this iteration, it has its candidate slot to schedule a prefetch ④.

The candidate iteration is scheduled in the *Prefetch Action Vector (PAV)* ⑤. Later, *State Recorder* will restore the state of the cache in the candidate iteration, and it will iterate from this point up to the current *"Iteration"*, updating the *Cache* to consider the prefetched checkpoint.

Unavoidable misses may still occur, in case there is no space in the PAV for the given candidate iteration ⑥. This means that during the actual execution, GPUZIP will synchronously bring the data from the host to the GPU, paying the cost of the slow transfers through the PCI-e.

Algorithm 1 The Prefetch Setup Algorithm

```
1:  Iteration ← 0
2:  repeat
3:    action, timestep ← GetAction()
4:    if action is "SAVE" then
5:      Cache.Push(timestep) ②
6:    else if action is "RESTORE" then
7:      if Cache Miss then
8:        Candidate ← IterOfLRU(Cache) + 1 ④
9:        if No Prefetch Scheduled in Candidate then
10:         Schedule(PAV, Candidate, timestep)
11:         StateRecorder.Restore(Candidate, Cache)
            ⑤
12:         Update Cache from the Candidate
13:       else
14:         Unavoidable Miss! ⑥
15:       end if
16:     else
17:       Cache.Touch(timestep) ③
18:     end if
19:   end if
20:   StateRecorder.Snapshot(Iteration, Cache) ①
21:   Iteration ← Iteration + 1
22: until action is not "TERMINATE"
```

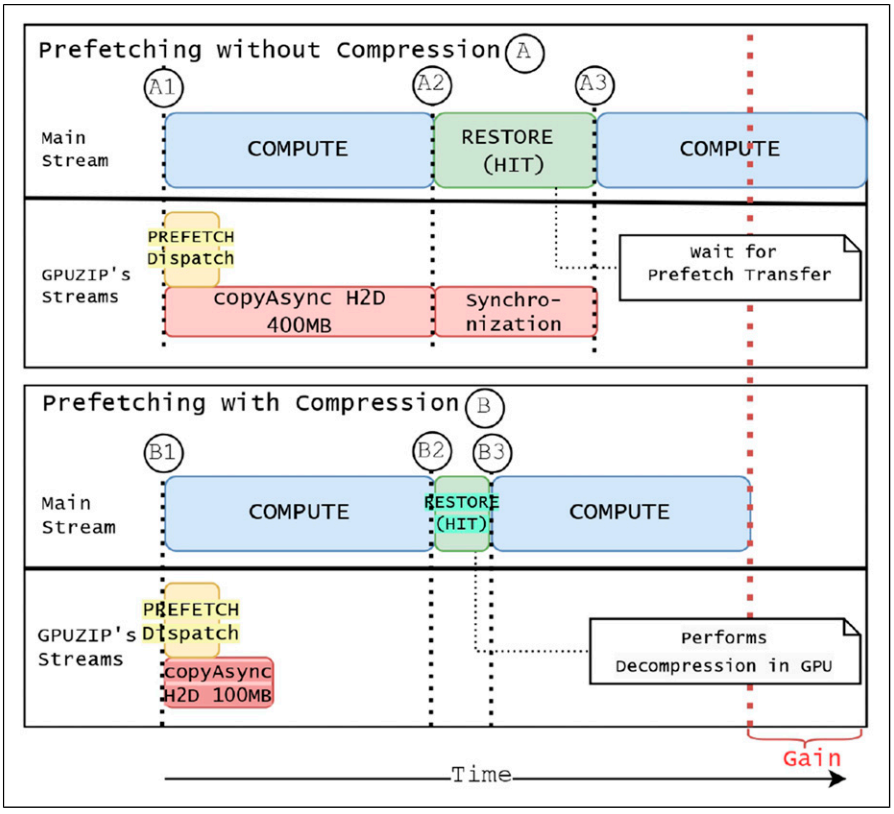

Figure 5. Checkpoint prefetching profiles with and without compression. Timeline "A" represents a scenario without compression, where a Prefetch Dispatch "A1" initiates the transfer of a 400 MB checkpoint from the host to GPU memory. The `RESTORE` action "A2" must synchronize with the transfer, postponing computation "A3". Timeline "B" shows the benefits of compression, where 4× smaller, compressed checkpoint is transferred "B1" and decompressed on the GPU "B2", avoiding synchronization and resulting in a performance gain. The dotted red line highlights the efficiency gain from compression.

5. GPUZIP's data compression

Introducing the prefetching mechanism in GPUZIP v2.0 significantly improves cache misses, achieving a 100% prevention rate in several experiments. However, it does not eliminate the PCIe bottleneck because the prefetched action may have insufficient time to transfer the checkpointing data from the host to the GPU memory before the `RESTORE` action.

Figure 5 illustrates in timeline Ⓐ a snippet of profiling of Prefetching without compression. In the main CUDA stream, a computation is happening (see (A1)), and in parallel, a Prefetch action is dispatched in GPUZIP's CUDA stream to retrieve the data to be used in the next `RESTORE`. The prefetch dispatch will asynchronously transfer a checkpoint of 400 *MB* from the host to the GPU memory, and the computation finishes before the transfer is complete. The next action is the checkpoint `RESTORE` (A2); since the transfer is still happening, the `RESTORE` action will need to wait for it, delaying the continuation of the program.

Re-visiting Figure 5, timeline Ⓑ demonstrates a snippet for the same program as before, but with compression applied. During the Prefetch Dispatch (B1), compressed checkpoint data – 4× smaller – is transferred from the host to the GPU memory, significantly reducing the transfer time. When the `RESTORE` action (B2) occurs, the data is already available to be decompressed in the GPU, which is a fast process, and then the following computation proceeds (B3). Since no data synchronization was needed, the result is an overall performance gain, illustrated by the "Gain" region in the figure.

Beyond improving `RESTORE` operations, compression also benefits `SAVE` actions. In cases where a Prefetch Dispatch uses a cache position in an ongoing `SAVE` transfer (device to host), the prefetch operation must wait for the `SAVE` to finish, creating a cascading delay that affects subsequent `RESTORE` actions. Transfer time is reduced by compressing the data during `SAVE`, mitigating delays and improving the efficiency of both `SAVE` and `RESTORE` operations. These optimizations, applied over thousands of `RESTORE` executions in real-world scenarios, significantly

improve overall performance and reduce computational bottlenecks.

GPUZIP adopts lossy compression to 32-bit floating-point data directly on the GPU and ensures that only the compressed data is transferred over the PCIe interface. During a `SAVE` operation, the data is compressed in GPU memory before being sent to the host. Similarly, a `RESTORE` operation transfers the compressed data from the host back to the GPU, where it is then decompressed.

The compression data flow is illustrated in Figure 4 in steps ①, ②, and ③. Step ① compresses and stores the data before sending it to the cache and host to save a checkpoint; the user calls GPUZIP's `SAVE` method, passing as an argument the pointer to the data used by the GPU kernels. GPUZIP compresses them and stores them directly in the *GPU Checkpoint Cache*.

Table 1. Calibration experiments examining configurations to balance compression ratio and quality. The columns are defined as follows: CR (AVG): Average Compression Ratio, higher values are better; PSNR: Peak Signal-to-Noise Ratio, higher values indicate better quality; MRE: Mean Relative Error, lower values are preferred; and SSIM: Structural Similarity Index Measure, values closer to one indicate greater similarity to the baseline.

Profile	CR AVG	PSNR	MRE	SSIM
bitcomp, delta = 1e-2	597.0	41.7	100.0	0.00
bitcomp, delta = 1e-4	206.0	65.4	59.8	0.45
bitcomp, delta = 1e-8	116.0	95.0	0.0	0.99
cuSZp, errBnd = 1e-2	13.7	41.4	100.0	0.00
cuSZp, errBnd = 1e-4	5.4	41.5	99.9	0.00
cuSZp, errBnd = 1e-8	2.4	41.7	99.9	0.00
cuZFP, maxBits = 2	15.7	48.0	83.7	0.71
cuZFP, maxBits = 4	7.9	68.7	12.1	0.98
cuZFP, maxBits = 8	3.9	98.0	0.4	0.99

The data stored in steps ② and ③ are decompressed only when needed by the GPU kernel to perform a `RESTORE` operation during a prefetch or a cache miss; the user calls GPUZIP's `RESTORE` method passing as argument the pointer to the data used by the GPU kernels and GPUZIP dumps decompression directly to the pointer location. This strategy of GPUZIP dumping the data directly to the pointer passed by the user reduces Device-to-Device copies, helping in the overall speedup.

There are multiple GPU-based lossy compressors available either as open-source projects or free-to-use libraries, each presenting different trade-offs between compression ratio, speed, and data quality (Di et al., 2025). Considering that users may want to experiment with different compression techniques depending on their application requirements, GPUZIP adopts a library-agnostic design to support the seamless integration of external compressors. The current GPUZIP implementation has been tested with three GPU-based compression libraries: cuZFP (Lindstrom, 2014), NVIDIA Bitcomp,[1] and cuSZp (Huang et al., 2023a). Although FZ-GPU (Zhang et al., 2023) and cuSZ (Tian et al., 2021) were also considered, FZ-GPU lacks APIs for integration, and cuSZ encountered memory issues.[2,3]

NVIDIA Bitcomp and cuSZp are error-bound lossy compressors, which compress up to a specified error limit between uncompressed and decompressed data. However, the error-bound approach does not guarantee a compression ratio as fixed-rate compressors can. cuZFP is a fixed-rate compressor, which may degrade the data more than the error-bound approach to reach the defined rate or compress less than an error-bound compression because it has already reached its target. Fixed-rate compressors can be beneficial in scenarios where memory is limited and the user needs to know exactly how much data to

Table 2. This table presents the results of calibration experiments with two shots, focusing on the time spent on PCI-e communication as well as the time spent on compression and decompression operations. The columns are defined as follows: **Transfer (d2h & h2d)**: This represents the total time measured for four GPUs during device-to-host (d2h) and host-to-device (h2d) communication, **Comp. & Decomp**: This indicates the total time spent on the compression and decompression of data across the same four GPUs.

Profile	Transfer d2h & h2d (hh:mm:ss)	Comp. & Decomp. (hh:mm:ss)
Baseline-No compress	1:16:10	-
bitcomp, delta = 1e-2	0:00:04	0:01:00
bitcomp, delta = 1e-4	0:00:31	0:01:04
bitcomp, delta = 1e-8	0:06:49	0:01:27
cuSZp, errBnd = 1e-2	0:03:53	0:06:14
cuSZp, errBnd = 1e-4	0:06:41	0:08:03
cuSZp, errBnd = 1e-8	0:15:22	0:12:51
cuZFP, maxBits = 2	0:02:45	0:02:51
cuZFP, maxBits = 4	0:05:33	0:03:32
cuZFP, maxBits = 8	0:11:16	0:04:01

allocate, allowing for savings in memory space through compression.

It is important to understand how lossy compression and the different libraries will affect the quality of the results in the context of a given application (Di et al., 2025). For our RTM application, a series of calibration experiments were conducted to define which libraries should be used in production. The experiments were performed migrating two shots with different configurations to find the balance between Compression Ratio (CR) and quality by measuring the Peak Signal-to-Noise Ratio (PSNR), Mean Relative Error (MRE), and the Structural Similarity Index Measure (SSIM). The data presented in Tables 1−2 only include results from the Marmousi3D dataset, as the observed patterns are consistent across all datasets. The complete results are available in the "DataWarmUp" directory within the repository referenced in Maltempi et al. (2024b).

Each compression library was configured in the calibration experiments using its respective parameters available through their APIs: Bitcomp uses the *delta*[4] parameter, which defines the maximum tolerated absolute error between the original and decompressed data; cuSZp uses the *errBound* parameter, configured in relative error mode, where the tolerated error is proportional to the data range (Huang et al., 2023a); and cuZFP uses the *maxBits*[5,6] parameter, which controls the number of bits allocated per value in fixed-rate mode.

The calibration experiments indicated that cuSZp underperformed compared to Bitcomp and cuZFP in balancing quality and compression ratio, as shown in Table 1. Bitcomp resulted in a mean relative error (MRE) of nearly zero and SSIM close to 1 with an error bound of 1×10^{-8}. cuZFP with a bit rate of 8 (resulting in a compression ratio of 4×) achieved a similar SSIM, but its MRE was slightly larger (0.4) than Bitcomp. In contrast, cuSZp exhibited significantly higher MRE (above 99%) and SSIM close to 0 across all tested error bounds, indicating poor accuracy and structural preservation.

To assess whether compression benefits the solution, we need to compare the time spent in the baseline's PCI-e transfers to the total time spent with the PCI-e transfer plus the time spent compressing and decompressing the data in the GPUZIP-based solution. In Table 2, the column labeled "Transfer d2h & h2d" shows that compression significantly reduces the time spent on PCI-e transfers. The overhead for compression and decompression is minimal compared to the reduction in PCI-e transfer time. For example, "cuZFP" with a maxBits of 8 took 4 minutes to compress and decompress and 11 minutes to transfer data through PCI-e; in contrast, the baseline scenario, which did not use compression, took an additional hour compared to cuZFP. In general, NVIDIA Bitcomp demonstrated the fastest compression and decompression operations, while cuSZp exhibited the slowest times.

The selected configurations for the following evaluation experiments were "cuZFP, maxBits = 8" and "bitcomp, delta = 1e-8" because they provided the best compressibility, quality, and speed results during compression and decompression. NVIDIA Bitcomp was chosen as the best library from the error-bound libraries evaluated; cuZFP is also interesting for scenarios where the compressed data size needs to be known beforehand, even though Bitcomp's performance generally surpasses cuZFP.

6. Experimental setup

The GPUZIP evaluation lays on Awave-3D, an RTM implementation designed to distribute shots across multiple GPUs to solve acoustic waves, developed under the guidance of geophysics experts. The experiments were run on a multi-GPU server with four NVIDIA Tesla V100-SXM2 32 GB GPUs and an Intel (R) Xeon (R) Gold 6240 CPU operating at 2.60 GHz. The server had 384 *GB* of RAM and operated with clang v15.0.1, CUDA version 11.2, and NVIDIA Driver version 525.60.13. The host RAM memory available for the checkpointing pool is up to 340 *GB*.

The three checkpoint algorithms discussed in Subsection 3.2 — *Revolve*, *zCut*, and *Uniform*—were used in the experiments. Each experimental profile for a checkpointing algorithm was compared to its respective baseline; for example, experiments using the *Revolve* algorithm were compared with the baseline implementation of *Revolve* that stores all checkpointing data in host memory. This work does not aim to determine the best checkpoint algorithm. The goal is to demonstrate how GPUZIP easily enables experimentation with different options for the same application, and how to explore different configurations of the library to achieve better performance for each algorithm.

All the experiments use GPUZIP's Prefetching mechanism with a selected *GPU Checkpoint Cache* size (ranging from two to six checkpoints) and may apply compression or not. The experiments without compression analyze the impact of the new v2.0 prefetching algorithm. The experiments combining prefetching and compression aim to find the configuration that best achieves the performance GPUZIP can deliver. The chosen compressor was cuZFP with a maxBits of 8 (compression ratio of 4) or NVIDIA's NVComp Bitcomp with an error bound of 1×10^{-8}. The compression parameters were determined during the warm-up phase, as described in Section 5.

The datasets used in the experiments are provided in the "Datasets" directory within the repository referenced in Maltempi et al. (2024b). The three datasets include:

(1) **Large Dataset**: An in-house dataset containing 3001 timesteps, with 15 × 14 shots distributed on a spatial grid of 500 × 500 × 500 (10, 000 × 10, 000 × 10, 000 m). Migrations were run with 192 shots.
(2) **Marmousi3D**: A 3D version of the Marmousi-II model (Martin et al., 2006), consisting of 6751 timesteps, with 13 × 3 shots distributed over a

grid of 351 × 901 × 301 (3, 510 × 9, 010 × 3, 010 m). Migrations were run with 36 shots.

(3) **Salt Dataset**: Based on the SEG/EAGE Salt and Overthrust models (Aminzadeh et al., 1997), comprising 3751 timesteps, with 13 × 13 shots distributed across a grid of 310 × 676 × 676 (6, 200 × 13, 520 × 13, 520 m). Migrations were run with 168 shots.

7. Experimental results

GPUZIP demonstrated significant acceleration of checkpointing by combining Prefetching and Compression techniques. The discussion of this section addresses the following key aspects:

(1) The performance improvements in GPUZIP 1.0 compared to 2.0 and the reasons behind these gains (Subsection 7.1).
(2) The achieved speedup by integrating GPUZIP into the baseline application (Subsection 7.2).
(3) The sensitivity of performance to GPU cache size (cache sensitivity) (Section 7.3).
(4) The bottlenecks associated with each checkpointing algorithm and the overhead introduced by GPUZIP (Subsection 7.4).
(5) GPUZIP v2.0 scalability evaluation with 2, 4, and six nodes (Subsection 7.5).
(6) Quality assessment when involving lossy compression (Subsection 7.6).

7.1. Prefetching mechanism comparison: v1.0 and v2.0

This section compares the efficiency of GPUZIP v1.0 and v2.0 due to the significant changes in the prefetching mechanism in the newer version.

GPUZIP v1.0 was evaluated using the *Revolve* algorithm with fixed size *GPU Buffer*, while GPUZIP v2.0was configured with a two-position cache to maintain similarity and comparability. No compression was applied during this analysis to directly assess the impact on the prefetching mechanism.

Table 3. The speedup ratio between GPUZIP v1.0 and v2.0 is analyzed by considering only the prefetch mechanism optimization and does not involve compression. The **Miss Rate** columns represent the number of misses in GPUZIP v1.0's *GPU Buffer* and GPUZIP v2.0's *GPU Checkpoint Cache* divided by the number of *Revolve*'s `RESTORE` actions.

Dataset	Speedup v1.0/v2.0	Miss rate v1.0 (%)	Miss rate v2.0 (%)
Large	1.52	5.93	0
Marmousi3D	1.54	11.27	0
Salt	1.53	8.45	0

As shown in Table 3, the column "Speedup v1.0/v2.0" represents the ratio of execution time improvement between v1.0 and v2.0. Across the three datasets, GPUZIP v2.0 consistently achieves a 1.5× speedup in execution times when compared to v1.0.

The improvement in GPUZIP v2.0 can be attributed to the elimination of cache misses, as shown in the "Miss Rate" column. GPUZIP v1.0 had miss rates ranging from 5.93% to 11.27%, whereas GPUZIP v2.0 achieved a 0% miss rate. GPUZIP v2.0 performs better due to its proactive strategy of removing the least recently used (LRU) checkpoint to prevent cache misses, making the checkpoint always available during `RESTORE` actions.

7.2. Overall speedup - GPUZIP v2.0

The experiments in this section measured the total runtime of Awave-3D from the beginning of the RTM execution to the end.

As illustrated in Figure 6, GPUZIP consistently delivered performance improvements across all profiles and datasets. Each line in the charts represents a checkpointing algorithm (*Revolve*, *zCut*, and *Uniform*). The *x*-axis shows the *GPU Checkpoint Cache* size configuration (2, 3, 4, 5, and 6 checkpoints), while the *y*-axis displays the speedup achieved. The bars are color-coded to indicate the use of GPUZIP's prefetching mechanism alone (blue), prefetching combined with cuZFP compression (orange), and prefetching combined with NVIDIA NVComp Bitcomp compression (green).

***Revolve*'s** performance without compression (blue bars) improves as cache sizes increase, as illustrated in Figure 6(a)–(c). For example, in the Marmousi dataset, the speedup increases from 2.3× with a cache size of 2 to 2.7× with a cache size of 3, then to 3.0× with a cache size of 4, and finally stabilizes at 3.2× for cache sizes of 5 and 6. This trend can be attributed to *Revolve*'s access pattern, which frequently retrieves different checkpoints while interspersing save operations. Larger caches help reduce evictions, enabling GPUZIP to store more checkpoints and avoid costly data transfers.

When compression is applied (orange and green bars), the cache size has a minimal impact, with the speedup varying only by 0.1×. This consistently occurs because compression mitigates synchronization time, as discussed in Section 5. For these datasets, two cache positions are enough when using compression, as prefetching can ensure that all necessary checkpoints are available ahead of time.

For ***zCut*** (Figure 6(d)–(f)), the speedup without compression remains steady, around 1.6× to 1.7×, regardless of cache size because *zCut* processes small intervals during the backward phase, where frequent saves and restores quickly overwhelm the cache, even at larger sizes. Compression significantly improves performance, with cuZFP achieving

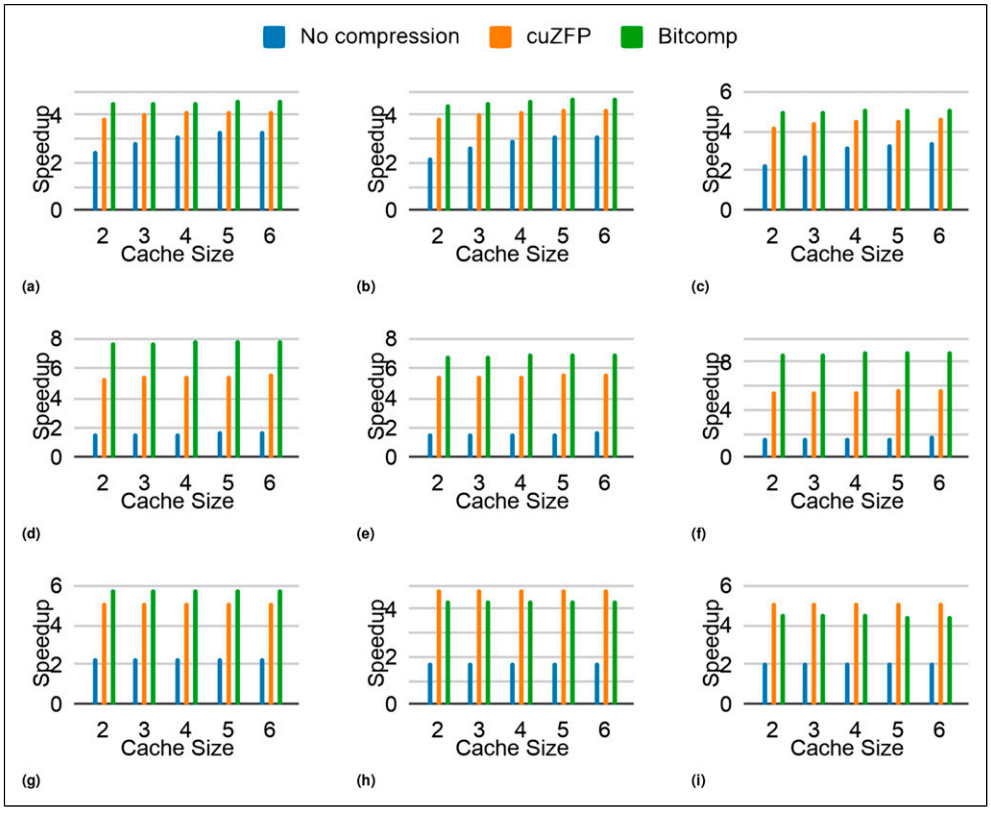

Figure 6. Overall Speedup achieved using GPUZIP v2.0 (with prefetching and compression) compared to the Baseline execution without GPUZIP (without prefetching and compression). (a) *Revolve*- Large Dataset, (b) *Revolve*- Marmousi3D Dataset, (c) *Revolve*- Salt Dataset, (d) *zCut*- Large Dataset, (e) *zCut*- Marmousi3D Dataset, (f) *zCut*- Salt Dataset, (g) *Uniform*- Large Dataset, (h) *Uniform*- Marmousi3D Dataset, (i) *Uniform*- Salt Dataset.

5.4× to 5.7× and Bitcomp reaching 6.9× to 8.9×. The Large dataset (Figure 6(d)) benefits the most from Bitcomp due to its simplicity and sparsity, which allow higher compression ratios, up to 600×.

For ***Uniform*** (Figure 6(g)–(i)), speedup without compression ranges from 1.8× to 2.3× among the three datasets, and it is not affected by cache size. Unlike *zCut*, *Uniform* restores checkpoints after long computational intervals, which means that additional cache memory provides little benefit. As expected, compression improves performance by reducing transfer times.

Compression has a particular impact on *Uniform*. By reducing the size of the checkpoints, *Uniform* can take more checkpoints within the same memory capacity of the host. In the current experimental configuration, cuZFP performs better with datasets with more timesteps (such as Marmousi3D and Salt) because it has a fixed compression rate of 4 and can take up to 4× more checkpoints. Although Bitcomp can achieve much higher compression ratios, its space requirements for storing checkpoints are unpredictable. Therefore, a conservative compression ratio was considered ranging from 2× to 2.6× for each data set.

7.3. Memory consumption vs speed evaluation

The reproducibility of the speedups presented in Subsection 7.2 depends heavily on the memory available in the host systems. Table 4 summarizes the memory consumption in both the host and the GPU to store checkpoints in each experiment, along with the total application runtime.

Each checkpointing algorithm determined the number of checkpoints to be stored according to its implementation to run the experiments. For example, *Revolve* required a checkpoint pool of 8, while *zCut* demanded a significantly larger pool of 149. *Uniform* tries to use all the memory available for the checkpointing pool, so experiments with compression allow more checkpoints to be taken using the same memory constraints.

Table 4 shows an example with Salt dataset of how exploration using GPUZIP techniques can expose different behaviors in each scenario. The choice of the best approach is up to the users, who will have their own cluster, application, and datasets. For example, in the experiment labeled "2,NoComp" (two checkpoints in the cache without compression), *Revolve* achieved faster performance than *zCut*

Table 4. Memory consumption for storing checkpoint data and execution time across various cache sizes and compression settings for the Salt dataset.

	Revolve				zCut				Uniform			
CacheSize, compress.	# Chk pts	GPU Mem (GB)	Host Mem (GB)	Spent time (h:m)	# Chk pts	GPU Mem (GB)	Host Mem (GB)	Spent time (h:m)	# Chk pts	GPU Mem (GB)	Host Mem (GB)	Spent time (h:m)
Baseline	8	0.0	16.5	35:26	149	0.0	329.80	47:31	149	0.0	329.40	29:34
2,NoComp.	8	4.1	16.5	14:52	149	4.1	329.80	29:55	149	4.1	329.40	13:51
2,Bitcomp	8	1.6	6.3	7:02	149	1.6	131.92	5:25	374	1.6	318.39	6:32
2,cuZFP	8	1.0	4.2	8:21	149	1.0	83.66	8:33	535	1.0	300.38	5:46
3,NoComp.	8	6.2	16.5	12:34	149	6.2	329.80	29:26	149	6.2	329.40	13:52
3,Bitcomp	8	2.4	6.3	6:59	149	2.5	131.92	5:23	374	2.4	318.39	6:32
3,cuZFP	8	1.6	4.2	8:02	149	1.6	83.66	8:30	535	1.6	300.38	5:46
4,NoComp.	8	8.2	16.5	11:02	149	8.2	329.80	28:08	149	8.2	329.40	13:51
4,Bitcomp	8	3.2	6.3	6:56	149	3.3	131.92	5:22	374	3.2	318.39	6:32
4,cuZFP	8	2.1	4.2	7:46	149	2.1	83.66	8:26	535	2.1	300.38	5:46
5,NoComp.	8	10.3	16.5	10:35	149	10.3	329.80	28:26	149	10.3	329.40	13:51
5,Bitcomp	8	4.0	6.3	6:55	149	4.1	131.92	5:20	374	4.0	318.39	6:33
5,cuZFP	8	2.6	4.2	7:39	149	2.6	83.66	8:20	535	2.6	300.38	5:46
6,NoComp.	8	12.4	16.5	10:23	149	12.4	329.80	27:19	149	12.4	329.40	13:51
6,Bitcomp	8	4.8	6.3	6:55	149	4.9	131.92	5:18	374	4.8	318.39	6:33
6,cuZFP	8	3.1	4.2	7:37	149	3.1	83.66	8:16	535	3.1	300.38	5:47

while consuming nearly 20× less memory. On the other hand, *Uniform* emerged as the fastest option under the same memory constraints as *zCut*. However, in the case of "2,Bitcomp", *zCut* outperformed *Revolve* and *Uniform* in speed while also benefiting from reduced memory usage due to compression.

Considering our cluster specifications and the dataset used in the experiments, *zCut* with Bitcomp and a cache size of two runs faster; however, users are encouraged to run preliminary experiments in a subset of their field to analyze the trade-off between memory consumption and speed. This preliminary testing helps identify the optimal configuration for their specific datasets and cluster resources.

7.4. Execution time breakdown and GPUZIP overhead

This section analyzes the reduction in blocking time in transferring checkpoint data. In this analysis, the blocking time comprises the time spent on all synchronous transfer operations. The overhead introduced by GPUZIP includes the time spent on compression, decompression, the Prefetch Setup Algorithm, and Prefetch Dispatch. The goal is to demonstrate how GPUZIP mitigates the bottlenecks inherent to each checkpointing algorithm.

Figure 7 presents three stacked bar charts, one for each checkpointing algorithm. The blue bars represent the time spent on `FORWARD` and `BACKWARD` computations (Awave-3D's GPU kernels). The blocking time for synchronous `SAVE` and `RESTORE` operations is shown in shades of orange, while GPUZIP's overhead is shown in green. These metrics were collected using two shots from the Salt dataset. The results for other data sets followed a similar pattern and are available in the "TimeBreakdown" directory within the repository referenced in Maltempi et al. (2024b).

GPUZIP significantly reduces the blocking time in `SAVE` and `RESTORE` operations for all checkpointing algorithms. The overhead introduced by GPUZIP (green bars) is notably less than the baseline blocking time for these operations.

Figure 7(a) shows the blocking times for ***Revolve***. GPUZIP decreases the blocking time for `SAVE` operations as the *GPU Checkpoint Cache* size increases for experiments without compression (see bars (2,-) and (3,-) in the figure. This improvement is due to GPUZIP's ability to store more checkpoints in the *GPU Checkpoint Cache*, reducing the frequency of host-to-device transfers due to misses. Moreover, when compression is applied, the `SAVE` and `RESTORE` operations' blocking time goes close to zero, and the GPUZIP overhead is irrelevant compared to the amount of blocking time it reduces in transferring.

For ***zCut*** (Figure 7(b)), increasing the *GPU Checkpoint Cache* size does not lead to noticeable improvements in the blocking time. *zCut* involves significantly more blocking transfers than *Revolve* and *Uniform*, due to intermediary checkpoints during the backward phase. Compression and prefetching are critical for reducing this bottleneck, as seen

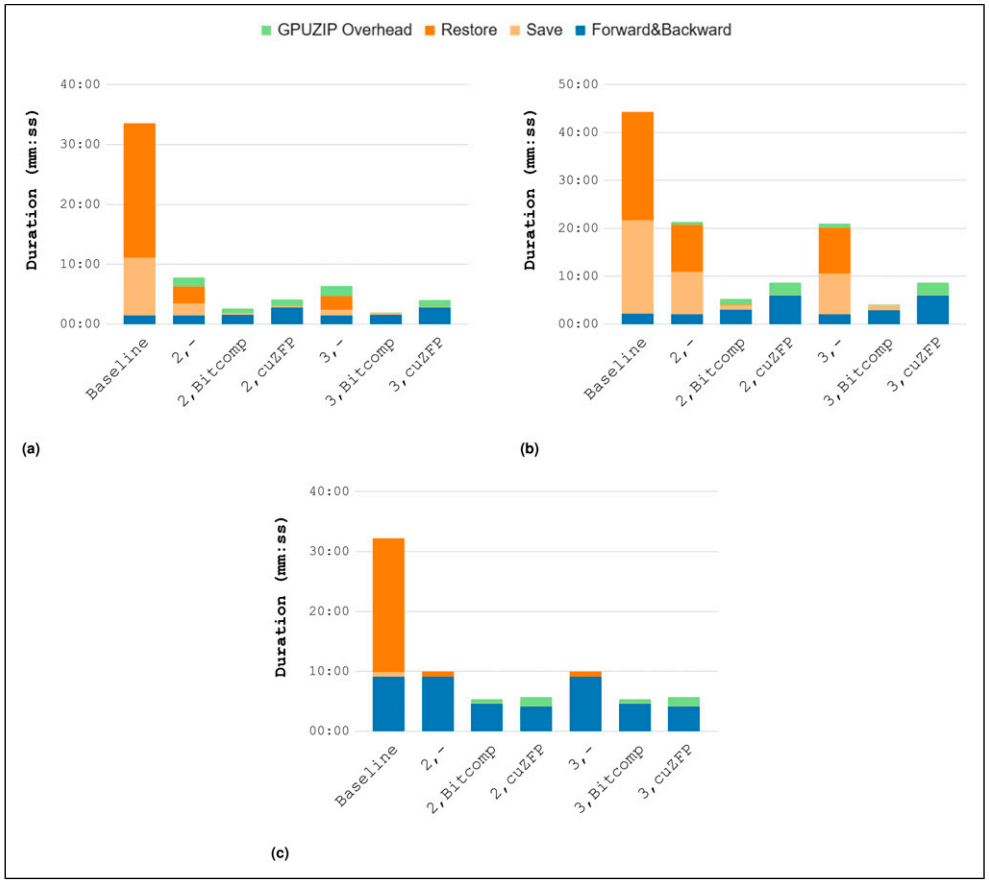

Figure 7. Execution time breakdown. The data corresponds to the Salt dataset for each checkpointing library. Label (N,L) means N cache positions, compressing using the L library. (N, -) means N cache positions with no compression. (a) *Revolve*, (b) *zCut*, (c) *Uniform*.

when using Bitcomp, which nearly eliminates all blocking time for `SAVE` and `RESTORE` operations.

As shown in Figure 7(c), ***Uniform*** exhibits distinct behavior compared to the other algorithms. Its blocking time for `SAVE` operations is significantly shorter than for `RESTORE` operations because the intervals between saves are long enough to allow asynchronous transfers to complete without causing delays. Furthermore, *Uniform* only saves checkpoints during the initial forward pass, with no intermediate saves, resulting in more recomputation than other algorithms. This is reflected in the higher blue bars in Figure 7(c).

Uniform's primary bottleneck in its baseline implementation lies in `RESTORE` operations. GPUZIP's prefetching mechanism alone significantly reduces this bottleneck. When combined with compression, GPUZIP not only further decreases the blocking time but also enables the creation of additional checkpoints by reducing the size of the checkpoint pool – notice the blue bars decreasing when compression is applied since less recomputation will be required. The orange and green bars remain very small, which leads to performance gains.

These results show how the combination of prefetching and compression introduced by GPUZIP significantly reduces the time spent on blocking communications in the application. The *Communication-Computation Ratio* (CCR) considering these blocking transfers goes from 0.05 to 4.48 for *Revolve*, from 0.05 to 2.67 for *zCut*, and from 0.39 to 10.75 for *Uniform*, when using Bitcomp for compression.

7.5. Multinode scalability

To evaluate the scalability of GPUZIP, we used the *Revolve* profile (Bitcomp with a *GPU Checkpoint Cache* size of 3), as it offers a balanced trade-off between memory usage and performance. This configuration also aligns with the one used to assess the first version of the library in our previous work Maltempi et al. (2024a). The scalability analysis involved comparing single-node execution to multi-node execution using 2, 3, 4, and 6 workers. The workload was distributed by assigning the shots to different workers, leveraging OpenMP Cluster (Yviquel et al., 2023), an open-source runtime designed to enable multinode scaling. All

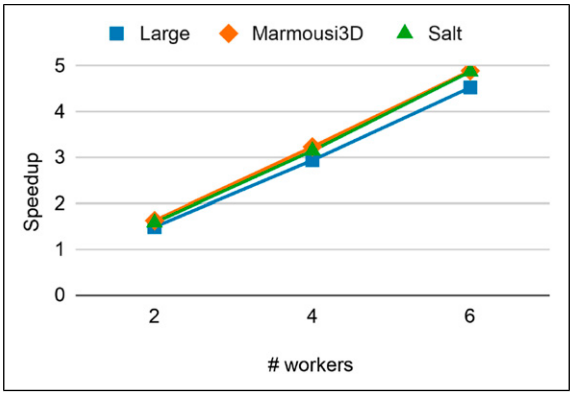

Figure 8. Scalability of GPUZIP with the *Revolve* profile (Bitcomp + *GPU Checkpoint Cache* size of 3) across varying numbers of workers (2, 3, 4, and 6) for the Large, Marmousi3D, and Salt datasets.

Table 5. Comparison of PSNR, relative error, and SSIM for cuZFP and Bitcomp.

Dataset	Metric	cuZFP	Bitcomp
Large	PSNR	105.98	138.06
	Relative error	4.94E-04	3.06E-07
	SSIM	0.99	0.99
Marmousi3D	PSNR	94.23	120.94
	Relative error	6.16E+01	1.31E-01
	SSIM	0.99	0.99
Salt	PSNR	78.01	135.82
	Relative error	1.62E+01	2.68E-05
	SSIM	0.99	0.99

nodes used in the experiment had identical specifications, as detailed in Section 6.

Figure 8 illustrates that the speedup improves with the number of workers. While an ideal scenario would see linear scaling, inter-node communication overheads slightly reduce the realized speedup compared to the theoretical maximum. Despite this, the experiment demonstrated a speedup of approximately 80% of the theoretical peak, a performance consistent across all benchmarks and comparable to the results from the library's earlier version.

7.6. Image quality assessment

A pertinent concern about the use of compression in scientific imaging is the potential impact on the resulting image quality when applying lossy compression. It is essential to balance compression ratios and quality to achieve significant speedups while minimizing the degradation of the final image. In this article, three metrics have been used to evaluate the quality of the final image: **PSNR** (*Peak Signal-to-Noise Ratio*), where higher values indicate better quality; **Relative error**, where lower values indicate higher fidelity; and **SSIM** (*Structural Similarity Index Measure*), which captures perceptual differences between images that ranges from 0 to 1, with values closer to one indicates higher fidelity. Table 5 compares those metrics across different datasets. Regarding PSNR, cuZFP consistently achieves slightly lower values than Bitcomp, with the largest deviation

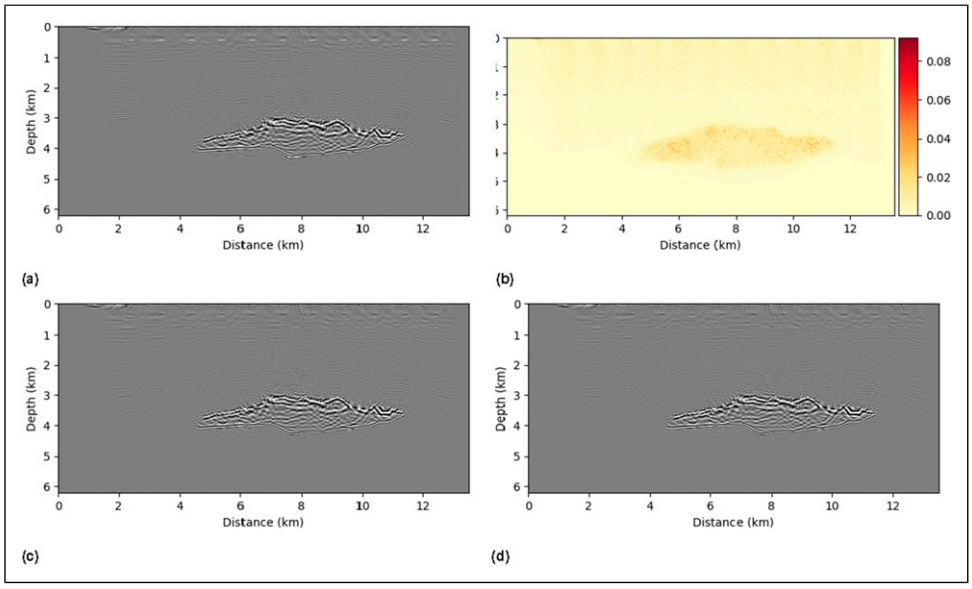

Figure 9. Comparison of migrated images for salt dataset. (a) Baseline, (b) Error Map with Bitcomp, (c) cuZFP, (d) Bitcomp.

observed in the Salt dataset (78.01 for cuZFP and 135.82 for Bitcomp). This is because bitcomp limits the error, not the compression ratio, as cuZFP. In contrast, bitcomp exhibits significantly lower relative errors in all datasets, indicating higher fidelity. Finally, SSIM values remain very close to one for all configurations and datasets, suggesting that both compressors preserve the structural content of the images despite their differences in PSNR and relative error.

Upon visual inspection, it is difficult for the naked eye to identify any differences between the Baseline and the results using compression. To conduct a more in-depth analysis, *Absolute error*, maps were created to measure the deviation between each image pixel, as shown in Figure 9(b) for the Salt dataset. In Salt's error map, it is possible to see some small discrepancies, represented in shades of red. Nevertheless, it is worth noting that the highest absolute error in this dataset was 0.08 within the −750 to 750 interval, indicating that the images produced by the Baseline and GPUZIP are almost identical. Other dataset's results can be evaluated and viewed in the reproducibility data available in Maltempi et al. (2024b).

8. Conclusion

This article presents GPUZIP v2.0, an enhanced version of the GPUZIP checkpoint compression library, featuring a redesigned prefetch management system and detection algorithm. The introduction of the GPU Checkpoint Cache enables fine-tuning GPUZIP configurations to suit specific checkpointing algorithms and application profiles, significantly enhancing its flexibility and adaptability.

The Reverse Time Migration (RTM) seismic imaging application served as a case study to demonstrate GPUZIP's ability to optimize checkpointing in adjoint scientific applications. A comprehensive evaluation methodology explored combinations of three checkpoint algorithms, three datasets, and three GPU data compression libraries. The results highlighted that the combination of Prefetching and Bitcomp compression was the most effective for the *Revolve* and *zCut* algorithms, while cuZFP's fixed ratio compression excelled for the *Uniform* algorithm by maximizing checkpoint storage and minimizing recomputation. GPUZIP enabled the exploration of various configurations, achieving a notable speedup of 5.2× in total execution time.

GPUZIP v2.0 has proven to be a valuable tool for optimizing checkpoint performance in HPC applications by considering the interplay of application requirements, input datasets, checkpoint algorithms, and compression techniques. Future work will extend GPUZIP's capabilities by integrating emerging compression libraries and checkpoint algorithms, such as cuSZp2 (Huang et al., 2024), and cuSZi (Liu et al., 2024), further enhancing its potential to accelerate scientific computing workloads.

Acknowledgements

The authors thank the Laboratório Nacional de Computação Científica (LNCC) and SENAI/CIMATEC for the computational resources.

Declaration of conflicting interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The authors disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This work was supported by Petrobras under grants [2018/00347-4] and [2022/00096-7], by the Fundação de Amparo à Pesquisa do Estado de São Paulo (FAPESP) under grant [2019/26702-8], and by Conselho Nacional de Desenvolvimento Científico e Tecnológico [302596/2022-4].

ORCID iDs

Thiago Maltempi https://orcid.org/0009-0007-9970-8197
Sandro Rigo https://orcid.org/0000-0002-9539-6874
Marcio Pereira https://orcid.org/0000-0003-1140-4513
Hervé Yviquel https://orcid.org/0000-0003-1214-3431
Jessé Costa https://orcid.org/0000-0002-2906-3588
Guido Araujo https://orcid.org/0000-0003-4869-5190

Data Availability Statement

The source code of both GPUZIP v1.0 and v2.0 is openly available on GitHub at: https://github.com/LSC-Unicamp/GPUZIP. The repository's main page provides documentation, tutorials, and links to access the specific releases of GPUZIP v1.0 and v2.0. We make the entire project available under the permissive MIT license to encourage its use and extension by the community. The data supporting the reproducibility and validation of the results presented in this article are available in the "Repositório de Dados de Pesquisa da Unicamp (REDU)". They are cited throughout the manuscript where relevant as Maltempi et al. (2024b). The repository includes the following resources: (a) Input datasets used for the experiments, including velocity models and seismic traces; (b) Profiling results obtained with NVIDIA Nsight Systems for performance evaluation; (c) Compression calibration data used to configure the compressors and analyze trade-offs; and (d) Quality assessment results, including metrics such as PSNR and MRE for the migrated images. Further information about the datasets and how to access them is detailed in the corresponding README files included in the repository.

Notes

1. https://developer.nvidia.com/nvcomp.

2. https://github.com/szcompressor/cuSZ/issues/76.
3. https://github.com/szcompressor/cuSZ/issues/70.
4. https://docs.nvidia.com/cuda/nvcomp/native_api.html\#_CPPv425bitcompCompressLossy_fp32K15bitcompHandle_tPKfPvf.
5. https://zfp.readthedocs.io/en/release0.5.4/modes.html\#c.zfp_stream.maxbits.
6. https://zfp.readthedocs.io/en/release0.5.4/modes.html\#mode-fixed-rate.

References

Aminzadeh F, Brac J and Kunz T (1997) *3-D Salt and Overthrust Models*. SEG/EAGE 3-D Modeling Series No. 1. Tulsa: Society of Exploration Geophysicists.

Barbosa CH and Coutinho AL (2023) Reverse time migration with lossy and lossless wavefield compression. In: 2023 IEEE 35th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD), Porto Alegre, Brazil, 17–20 October 2023, IEEE, pp. 192–201.

Baysal E, Kosloff D and Sherwood J (1983) Reverse time migration. *Geophysics* 48(11): 1514–1524.

Di S, Liu J, Zhao K, et al. (2025) A survey on error-bounded lossy compression for scientific datasets. https://arxiv.org/abs/2404.02840

Dmitriev M, Tonellot T, AlSalem H, et al. (2022) Error-bounded lossy compression in reverse time migration. *Sixth EAGE High Performance Computing Workshop*. Utrecht, Netherlands: EAGE Publications BV, 2022, 1–5.

Git (2025) Gpuzip. https://github.com/LSC-Unicamp/GPUZIP (Accessed: 2025-04-07).

Griewank A and Walther A (2000) Algorithm 799: Revolve: an implementation of checkpointing for the reverse or adjoint mode of computational differentiation. *ACM Transactions on Mathematical Software* 26(1): 19–45.

Hennessy JL and Patterson DA (2017) *Computer Architecture, Sixth Edition: A Quantitative Approach*, chapter 2. 6th edition. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc, pp. 81–84.

Huang Y, Di S, Yu X, et al. (2023a) cuSZp: an ultra-fast GPU error-bounded lossy compression framework with optimized end-to-end performance. In: SC23: International Conference for High Performance Computing, Networking, Storage and Analysis, Denver, CO, USA, 11–17 November 2023, pp. 1–13.

Huang Y, Zhao K, Di S, et al. (2023b) Towards improving reverse time migration performance by high-speed lossy compression. In: 2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid), Bangalore, India, 01–04 May 2023, pp. 651–661.

Huang Y, Di S, Li G, et al. (2024) CUSZP2: a GU lossy compressor with extreme throughput and optimized compression ratio. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis, SC '24, Atlanta, GA, USA, 17–22 November 2024, IEEE Press.

Kukreja N, Hückelheim J, Louboutin M, et al. (2019) Combining checkpointing and data compression to accelerate adjoint-based optimization problems. In: *Euro-Par'2019*. Cham: Springer, pp. 87–100.

Lindstrom P (2014) Fixed-rate compressed floating-point arrays. *IEEE Transactions on Visualization and Computer Graphics* 20(12): 2674–2683.

Liu J, Tian J, Wu S, et al. (2024) cusz-i: high-ratio scientific lossy compression on gpus with optimized multi-level interpolation. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis, SC '24, Atlanta, GA, USA, 17–22 November 2024, IEEE Press.

Louboutin M, Lange M, Luporini F, et al. (2019) Devito (v3.1.0): an embedded domain-specific language for finite differences and geophysical exploration. *Geoscientific Model Development* 12(3): 1165–1187.

Luporini F, Louboutin M, Lange M, et al. (2020) Architecture and performance of devito, a system for automated stencil computation. *ACM Transactions on Mathematical Software* 46(1): 1–28.

Maltempi T, Rigo S, Pereira M, et al. (2024a) Combining compression and prefetching to improve checkpointing for inverse seismic problems in gpus. In: Carretero J, Shende S, Garcia-Blas J, et al. (eds) *Euro-Par 2024: Parallel Processing*. Cham: Springer Nature Switzerland, pp. 167–181.

Maltempi T, Rigo S, Yviquel H, et al. (2024b) Replication data for GPUZIP v2.0: accelerating checkpointing on GPUs with prefetching and compression.

Margetis AS, Papoutsis-Kiachagias EM and Giannakoglou KC (2021) Lossy compression techniques supporting unsteady adjoint on 2D/3D unstructured grids. *Computer Methods in Applied Mechanics and Engineering* 387: 114152.

Margetis ASI, Papoutsis-Kiachagias EM and Giannakoglou KC (2023) Reducing memory requirements of unsteady adjoint by synergistically using check-pointing and compression. *International Journal for Numerical Methods in Fluids* 95: 23–43.

Martin G, Wiley R and Marfurt KJ (2006) Marmousi2: an elastic upgrade for Marmousi. *The Leading Edge* 25(2): 156–166.

Maurya A, Rafique MM, Cappello F, et al. (2023a) Towards efficient I/O pipelines using accumulated compression. In: 2023 IEEE 30th International Conference on High Performance Computing, Data, and Analytics (HiPC), Goa, India, 18–21 December 2023, IEEE.

Maurya A, Rafique MM, Tonellot T, et al. (2023b) GPU-enabled asynchronous multi-level checkpoint caching and prefetching. In: HPDC '23: The 32nd International Symposium on High-Performance Parallel and Distributed Computing, New York, NY, USA, June 16–23, 2023, ACM.

Rigon P, Schussler B, Sardinha A, et al. (2024) Harnessing data movement strategies to optimize performance-energy efficiency of oil & gas simulations in HPC. *Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics) 14802 LNCS*. Cham, Switzerland: Springer, 211–225.

Shen J, Wu Y, Okita M, et al. (2022) Accelerating gpu-based out-of-core stencil computation with on-the-fly compression. In: *PDCAT'22*. Cham: Springer, pp. 3–14.

Shen J, Long L, Deng X, et al. (2023) A compression-based memory-efficient optimization for out-of-core gpu stencil computation. *The Journal of Supercomputing* 79: 11055–11077.

Tian J, Di S, Yu X, et al. (2021) Optimizing error-bounded lossy compression for scientific data on GPUs. In: *2021 IEEE International Conference on Cluster Computing (CLUSTER)*. Los Alamitos, CA, USA: IEEE Computer Society, pp. 283–293.

under review (2024) zcut: an optimization algorithm for checkpointing computional graphs. Under review.

Yviquel H, Pereira M, Francesquini E, et al. (2023) *The Openmp Cluster Programming Model*.

Zhang B, Tian J, Di S, et al. (2023) FZ-GPU: a fast and high-ratio lossy compressor for scientific computing applications on GPUs. In: *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing, HPDC '23*. New York, NY, USA: Association for Computing Machinery, pp. 129–142.

Author biographies

Thiago Maltempi is a Master's student in Computer Science at the University of Campinas (UNICAMP), advised by Professor Sandro Rigo and co-advised by Professor Guido Araujo. He earned his Analysis and Systems Development degree from Faculdade de Tecnologia de São Paulo (FATEC) in 2014. With over a decade of experience in software engineering, he has worked in the health-tech industry and contributed to various R&D initiatives. His academic research focuses on accelerating GPU programs using data compression and prefetching techniques. His research interests include computer systems, high-performance computing, and parallel computing.

Prof. Sandro Rigo received his Ph.D. in Computer Science from the University of Campinas (UNICAMP) in 2004 for his work on the ArchC Architecture Description Language. He is currently an Associate Professor at the Institute of Computing at UNICAMP. Prof. Rigo was the Chair of the B.Sc. Program in Computer Science from 2009 to 2011. He also served as the Information Technology and Communication Coordinator and Director of the Computing Centre at UNICAMP from 2017 to 2021. He co-received best paper awards at the Symposium on Computer Architecture and High Performance Computing (SBAC-PAD 2004), the Workshop on Computer Architecture and High Performance Systems (WSCAD 2012), and the Brazilian Symposium on Information and Computer System Security (SBSEG 2016). His current research interests include various aspects of computer systems design, such as the parallelization and acceleration of machine learning models in heterogeneous distributed systems, parallel programming models in HPC clusters, energy efficiency in computer systems, and code generation and optimization for performance, security, and energy consumption.

Dr. Marcio Pereira is a Senior Compiler Engineer with a Ph.D. in Compiler Technology for Transactional Memory from UNICAMP and the University of Alberta (Edmonton, Canada). He brings over 30 years of software development experience, including 12 years specializing in LLVM code generation and parallelization. His expertise spans neuromorphic compiler design for the Meta Glow and TensorFlow XLA toolchains and the MLIR Transform dialect. Dr. Pereira has authored numerous papers in leading international conferences and journals and received the IEEE SBAC-PAD 2017 Best Paper Award. He has led optimization projects for industry leaders such as Samsung, LG, IBM, and Microsoft, showcasing his ability to deliver innovation under tight deadlines. He has also held senior management roles at CPqD (Center for Research and Development in Telecommunications), with a strong focus on leadership and client engagement. He is currently conducting pioneering research at CEPETRO (Center for Petroleum Studies), UNICAMP.

Prof. Hervé Yviquel received his Ph.D. in Computer Science from the University of Rennes 1 (France) in 2013, following collaborative research between IRISA/Inria and INSA de Rennes. He has been a faculty member at the Institute of Computing at UNICAMP since 2020 and is part of the Laboratory of Computer Systems (LSC). After a year as a research engineer at INSA de Rennes, he moved to Brazil to work as a postdoctoral researcher at UNICAMP's Center for Computing in Engineering and Sciences (CCES). His primary research interests are in parallel computing, including compilers, scheduling, programming models, cloud technologies, and high-performance computing.

Gustavo Leite received his Bachelor's degree in Computer Science from São Paulo State University (UNESP) in 2016 and completed his Master's in Computer Science from the same institution in 2019. During his Master's program, Gustavo visited the University of Alberta (Canada) under the co-supervision of Professors José Nelson Amaral and Guido Costa Souza de Araújo. His research interests include high-performance computing, compilers, parallel computing, and machine learning.

Prof. Orlando Lee holds a Bachelor's degree in Computer Science (1991), a Master's degree in Applied Mathematics (1994), and a Ph.D. in Applied Mathematics (1999), all from the University of São Paulo. He is a faculty member at the University of Campinas (UNICAMP). His expertise lies in Computer Science and Combinatorics, emphasizing Graph Theory. His research focuses on combinatorial optimization, graph theory, and approximation algorithms.

Prof. Jessé Costa holds a Bachelor's degree in Physics (1983), a Master's in Geophysics (1987), and a Ph.D. in Geophysics (1994), all from the Federal University of Pará. He is currently a Full Professor at the Institute of Geosciences at the Federal University of Pará, working in both the Faculty of Geophysics and the Graduate Program in Geophysics. His primary research interests are in Applied Seismology, focusing on wave propagation in anisotropic media, seismic modeling, seismic imaging, and velocity analysis.

Prof. Guido Araújo received a Ph.D. in Electrical Engineering from Princeton University in 1997. He is a Full Professor of Computer Science and Engineering with UNICAMP. His current research interests include code optimization, parallelizing compilers, and compiling for accelerators, which are explored in close cooperation with industry partners. He has published over 120 scientific papers and received five best paper awards. He was awarded two Inventor of the Year and two Zeferino Vaz Research Awards from UNICAMP. His students received two Best Ph.D. Thesis Awards from the Brazilian Computing Society. He was a member of the technical advisory board of Eldorado and SIDI R&D Institutes and the CI-Brasil Council at the Brazilian Ministry of Science and Technology. He co-founded the companies Kryptus and Idea! and is currently a member of the Editorial Board of IEEE MICRO.